VICTOR. A

OPEN CLASSROOM

PROJET 9

DÉVELOPPEZ UNE PREUVE DE CONCEPT

CONTEXTE ET OBJECTIFS DU PROJET

- **Contexte** : Dans le cadre du processus de recrutement, il est demandé de réaliser un test technique. Il est également attendu de mener une veille et d’identifier une méthode plus récente

- **Objectif** : Reprendre et consolider le Projet 8 d’OpenClassrooms, puis intégrer les éléments du Projet 9 afin d’améliorer la qualité, la performance et la modernité de la solution développée.

MISE EN PLACE

🔧 Environnement & Configuration

Paramètre	Valeur
Framework	TensorFlow 2.16.1
GPU	Non
Project root	`C:\Users\vicau\P8OC`
Train images	580
Train masks	580
Notebook d'initialisation	OK

📊 Données, Qualité & Pipeline

Catégorie	Train	Validation	Test	Commentaire
Samples (images)	580	262	10	Répartition initiale
Masques	580	262	10	Alignement images / masques
Paires corrompues filtrées	0	0	0	Aucune donnée invalide
Samples après filtrage	580	262	10	Dataset inchangé
Batches	145	65	2	Selon batch size
DataGenerators	OK	OK	OK	Pipeline fonctionnel

L'environnement de travail est correctement configuré et le pipeline de données est pleinement opérationnel. La répartition des jeux de données est cohérente, sans données corrompues, ce qui constitue une base saine pour l'entraînement et l'évaluation du modèle.

MISE EN PLACE

Dictionnaire de correspondance – Cityscapes (34 → 8 classes)

ID Cityscapes	Classe d'origine	Classe simplifiée	ID simplifié
0–6	Unlabeled / Ego / Static / Ground…	Background	0
7	Road	Road	1
8	Sidewalk	Sidewalk	2
9–10	Parking / Rail track	Background	0
11	Building	Building	3
12–16	Wall / Fence / Bridge / Tunnel	Other construction	4
17–20	Pole / Traffic light / Traffic sign	Object	5
21–22	Vegetation / Terrain	Vegetation	6
23–33	Sky / Person / Vehicle / Rider	Vehicle / Agents	7

Synthèse du mapping et impact sur les masques

Élément	Valeur
Nombre de classes initiales	34
Nombre de classes simplifiées	8
Valeurs uniques (masque original)	16 labels observés
Valeurs uniques (masque simplifié)	7 labels observés
Objectif du mapping	Réduction de la complexité et stabilité de l'apprentissage

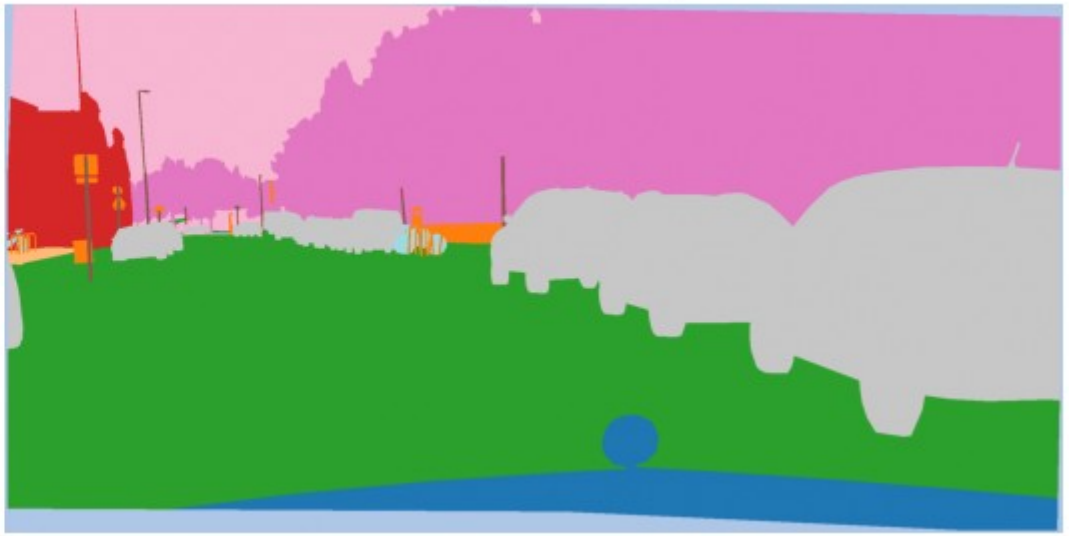
Le regroupement des 34 classes Cityscapes en 8 catégories sémantiques permet de réduire la complexité des masques tout en conservant les informations essentielles à la compréhension de la scène routière.

La diminution du nombre de labels observés dans les masques confirme l'efficacité du mapping et facilite l'apprentissage du modèle en limitant la dispersion des classes.

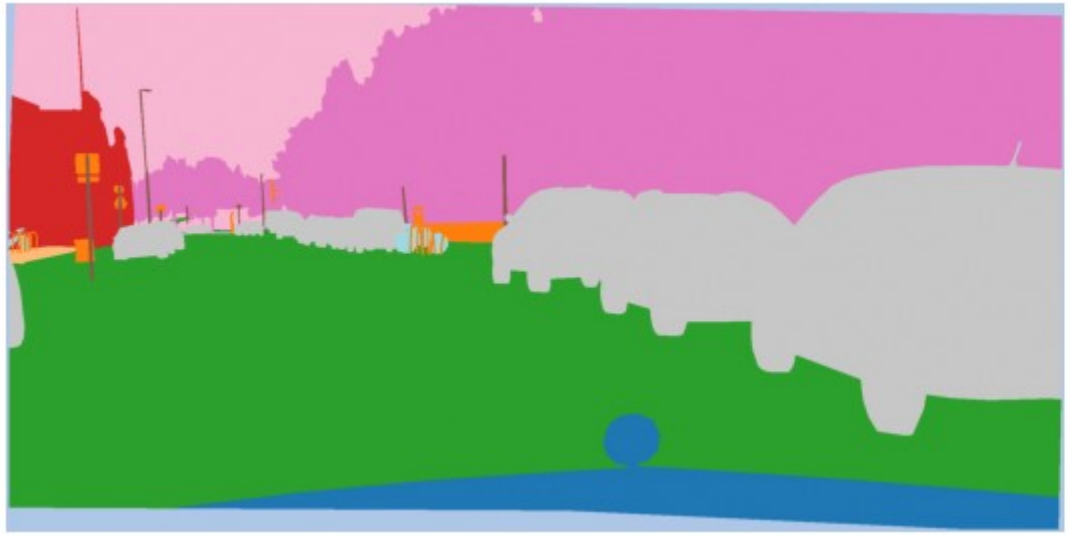
MISE EN PLACE

Image réelle

Mask (IDs 0 à 33)



Mask original - 34 classes



Mask remappé - 8 classes

PRE-PROCESSING

Construction du jeu de données – Pipeline de préparation

Phase	Étape	Description technique	Méthode / implémentation	Impact sur le pipeline
Analyse	Listing des villes	Exploration de la structure Cityscapes afin d'identifier les villes disponibles par *split*	Parcours des dossiers `leftImg8bit`	Garantit la cohérence entre données brutes et ensembles d'entraînement
Sélection	Sous-ensemble	Sélection volontairement réduite d'un échantillon représentatif du jeu de données	3 villes *train* / 1 *val* / 1 *test*	Accélère les phases d'exploration et de prototypage
Structure	Dossiers cibles	Création d'une arborescence dédiée séparant images et masques	`data/processed/images` / `masks`	Simplifie le chargement des données et le fonctionnement du DataLoader
Nettoyage	Réinitialisation du dataset	Suppression complète des données existantes avant reconstruction	`shutil.rmtree`	Évite les incohérences et les doublons
Préparation	Appariement image–masque	Association stricte image / masque basée sur la convention de nommage	`leftImg8bit` → `gtFine_labelIds`	Garantit l'alignement pixel par pixel
Construction	Copie contrôlée	Copie exclusive des paires image–masque validées	Vérification de l'existence des fichiers	Assure l'intégrité du jeu de données final
Résultat	Dataset prêt	Données organisées et exploitables par *split*	*train* / *val* / *test*	Dataset directement utilisable pour l'entraînement

Ce préprocessing permet de construire un sous-ensemble Cityscapes cohérent à partir des données brutes. La sélection contrôlée des villes réduit la taille du dataset tout en conservant sa diversité. Le dataset obtenu est directement exploitable par le DataLoader et les modèles de segmentation.

DATALOADER

DataLoader – Gestion des labels et chargement des données

Phase	Étape	Description	Détails / Paramètres clés
Labels	Mapping des labels	Regroupement des classes Cityscapes pour réduire la granularité	**34 → 8 classes**
Labels	Remapping des masques	Conversion des labels d'origine vers les classes simplifiées	`remap_mask(mask)`
Chargement	Générateur de données	Générateur Keras personnalisé pour le chargement par batch	`tf.keras.utils.Sequence`
Chargement	Images & masques	Lecture des images RGB et des masques bruts	`cv2.imread`, `IMREAD_UNCHANGED`

DataLoader – Prétraitement et sortie des batchs

Phase	Étape	Description	Détails / Paramètres clés
Prétraitement	Redimensionnement	Uniformisation des dimensions image et masque	**256 × 512**
Prétraitement	Augmentations	Transformations pour améliorer la robustesse du modèle	**Flip**, crop, rotation *(train)*
Prétraitement	Normalisation	Mise à l'échelle des pixels pour stabiliser l'apprentissage	**/ 255.0**
Sortie	Encodage & batchs	Encodage multi-classes et génération des lots	**One-hot (8) — X / y**

Le DataLoader centralise le chargement, le prétraitement et la mise en forme des données avant l'entraînement. La simplification des labels et le remapping des masques réduisent la complexité du problème tout en conservant l'information sémantique essentielle. Les batchs produits sont directement compatibles avec les modèles de segmentation.

GITHUB ACTION

CI/CD – GitHub Actions (API & Streamlit)

Étape	Nom / Action	Description	Détails clés
Déclencheur	`on: push` / `workflow_dispatch`	Lancement automatique du workflow sur la branche principale	**main**
Job 1 : Tests	Pytest	Vérification de la stabilité du code avant déploiement	Ubuntu, Python **3.13**, `pytest -q`
Job 2 : Déploiement API	Deploy to Heroku	Déploiement de l'API FastAPI (désactivé volontairement)	Slug > **500 Mo**, dernière version conservée
Job 3 : Déploiement Streamlit	Deploy to Heroku	Déploiement automatique de l'application front	**p8oc-streamlit**
Point d'entrée unique	`run.py`	Sélection dynamique du service à lancer	`APP_MODE=api` / `APP_MODE=streamlit`
Mode API	Uvicorn	Lancement de l'API FastAPI	`main:app`
Mode Streamlit	Streamlit	Lancement de l'interface web	`streamlit_app.py`

Le workflow GitHub Actions automatise l'exécution des tests à chaque mise à jour de la branche principale. Cette étape garantit la stabilité du code avant toute tentative de déploiement.

Le déploiement de l'API est volontairement figé en raison des contraintes de taille sur Heroku, tandis que l'application Streamlit reste automatiquement mise à jour. Le script **run.py** permet de piloter dynamiquement le service lancé selon le mode d'exécution.

MODÈLES CHOISIS

Présentation des architectures de segmentation évaluées

Modèle	Description technique	Signification / principe clé
UNet + VGG16	UNet intégrant VGG16 pré-entraîné comme encodeur, permettant une extraction progressive et hiérarchique des informations visuelles.	VGG (Visual Geometry Group) : 16 correspond au nombre de couches de convolution, favorisant une représentation visuelle riche, stable et structurée.
UNet + EfficientNetV2	UNet intégrant EfficientNetV2 comme encodeur, optimisé pour un apprentissage plus rapide et une meilleure efficacité computationnelle.	EfficientNetV2 repose sur un *scaling* optimisé pour l'entraînement et l'utilisation de blocs convolutifs efficaces, améliorant le compromis entre performance et coût de calcul.

Le modèle **VGG16** constitue le point de départ du projet, en tant qu'architecture relativement lourde issue du Projet 8. Dans le cadre du Projet 9, l'objectif est d'explorer une architecture plus moderne, datant de moins de cinq ans, tout en s'inscrivant dans une logique de légèreté proche de MobileNet. **EfficientNetV2**, introduit en 2021, répond à ces critères en offrant un meilleur compromis entre modernité, efficacité computationnelle et performances.

EFFICIENTNETV2

Slide 1 — EfficientNetV2 : théorie & principes clés

Axe d'analyse	EfficientNetV2 (principe)	Signal principal	Lecture / positionnement
Modernité	Architecture introduite en **2021** (famille EfficientNetV2)	**Modèle récent**	Cohérent avec l'objectif "< 5 ans" et une approche actuelle
Training-aware design	Conçu en intégrant la **vitesse d'entraînement** et l'efficacité	**↑ vitesse / ↓ coût**	Vise un compromis perf/coût plus réaliste que les backbones classiques
Blocs efficaces	Usage de **MBConv** et **Fused-MBConv** selon les étages	**↑ efficacité compute**	Meilleure exploitation des convolutions, surtout sur régimes contraints
Scaling optimisé	Mise à l'échelle pensée pour garder de bonnes perf sans explosion de complexité	**perf/complexité**	Architecture scalable sans surdimensionner inutilement
Pré-entraînement	Backbone pré-entraîné utilisé comme **extracteur de features**	**↑ transfert**	Démarrage plus stable, convergence plus rapide à budget identique

EfficientNetV2 est une architecture récente, introduite en 2021, retenue pour s'appuyer sur un modèle moderne et cohérent avec les pratiques actuelles. Sa conception *training-aware* intègre directement les contraintes liées à la vitesse d'entraînement et au coût de calcul, afin de viser un compromis réaliste entre performance et efficacité. L'utilisation de blocs MBConv et Fused-MBConv, associée à une mise à l'échelle optimisée et à un backbone pré-entraîné, permet de maîtriser la complexité du modèle tout en assurant une convergence plus rapide et plus stable.

EFFICIENTNETV2

Slide 2 — EfficientNetV2 en segmentation (rôle d'encodeur UNet)

Bloc / étape	Rôle	Entrée → Sortie	Signal principal	Contrôle / validation
Prétraitement	Standardiser le format d'entrée	Image RGB → tenseur normalisé	**stabilité**	Vérif dimensions / type / range
Encodeur EfficientNetV2	Extraire des features hiérarchiques	tenseur → maps multi-niveaux	↑ **contexte**	Cohérence des shapes par étage
Multi-résolution	Capturer du global + détails	niveaux profonds / peu profonds	**détails + structure**	Points de collecte alignés (skip)
Skip connections (UNet)	Réinjecter l'info spatiale fine	features encodeur → décodeur	↑ **précision locale**	Matching des résolutions / concat
Stratégie d'entraînement	Limiter le coût et stabiliser	backbone **gelé** + décodeur appris	↓ **compute** / ↑ **contrôle**	Surveiller convergence + overfit

Le pipeline commence par un prétraitement visant à standardiser les images d'entrée et à garantir la stabilité de l'apprentissage. EfficientNetV2 est ensuite utilisé comme encodeur afin d'extraire des caractéristiques hiérarchiques sous forme de cartes multi-résolution, permettant de capter à la fois le contexte global et les détails fins. Les *skip connections* du UNet réinjectent ces informations vers le décodeur pour améliorer la précision spatiale, tandis que le gel du backbone permet de réduire le coût de calcul et de stabiliser la convergence du modèle.

EFFICIENTNETV2

Slide 3 — Méthode mathématique (évaluation) : Dice, IoU, Accuracy

Indicateur	Formule	Ce que ça mesure	Exemple simplifié	Lecture / usage
Dice	$\frac{2\lvert A \cap B\rvert}{\lvert A\rvert + \lvert B\rvert}$	Recouvrement entre prédiction et vérité terrain	10 pixels réels, 12 prédits, 8 corrects → Dice ≈ 0,73	Mesure à quel point la prédiction recouvre bien la zone attendue, en restant tolérante aux petits écarts
IoU	$\frac{\lvert A \cap B\rvert}{\lvert A \cup B\rvert}$	Chevauchement strict	Union = 14 → IoU ≈ 0,57	Plus exigeant que Dice, pénalise davantage les faux positifs et faux négatifs
Pixel Accuracy	$\frac{\text{pixels correctement prédits}}{\text{pixels totaux}}$	Exactitude globale	Dépend fortement du fond dominant	Facile à lire mais peut être trompeur en segmentation, à interpréter avec Dice et IoU
Score synthétique	$\frac{\text{Dice} + \text{IoU} + \text{Accuracy}}{3}$	Vision agrégée	Combine les trois métriques	Donne une vue d'ensemble équilibrée sans dépendre d'un seul indicateur

Les performances sont évaluées à l'aide de métriques complémentaires. Le Dice mesure le recouvrement global, l'IoU fournit une évaluation plus exigeante des erreurs, et la pixel accuracy donne une vue générale souvent dominée par le fond. Leur combinaison dans un score synthétique permet de comparer efficacement les modèles dans le cadre de la preuve de concept.

EFFICIENTNETV2

Indicateur	Formule	Ce que ça mesure	Exemple simplifié	Lecture / usage
Scaling du modèle	$\text{Taille du modèle} \propto (\text{profondeur} \times \text{largeur} \times \text{résolution})$	Complexité globale du réseau	Modèle plus grand = plus de couches, plus de canaux, images plus grandes	EfficientNetV2 fait croître le modèle de façon équilibrée, sans surdimensionner un seul paramètre
Principe clé	Scaling conjoint	Répartition du budget de calcul	Éviter « plus profond seulement »	Améliore l'efficacité réelle à budget constant

L'architecture EfficientNetV2 repose sur un principe de *scaling conjoint*, dans lequel la complexité du modèle est répartie de manière équilibrée entre la profondeur, la largeur et la résolution d'entrée. Cette approche permet de faire croître le modèle sans surdimensionner un seul paramètre, en optimisant l'utilisation du budget de calcul. En pratique, ce mécanisme vise à améliorer l'efficacité réelle du réseau, en particulier dans des contextes contraints en ressources, comme une preuve de concept exécutée sur CPU.

VERSUS THÉORIQUE

Comparaison théorique — UNet + VGG16 vs UNet + EfficientNetV2

Axe d'analyse	UNet + EfficientNetV2	UNet + VGG16	Lecture / impact sur la décision
Modernité de l'architecture	Architecture récente (2021), conçue avec des contraintes d'efficacité d'entraînement	Architecture historique (2014), initialement pensée pour la classification	EfficientNetV2 s'inscrit dans une logique d'évolution technologique cohérente avec le Projet 9
Philosophie de conception	Approche *training-aware* intégrant vitesse et coût de calcul	Approche classique basée sur la profondeur du réseau	EfficientNetV2 vise une efficacité réelle à l'entraînement, au-delà de la seule précision
Complexité et coût de calcul	Modèle plus compact, meilleure utilisation du budget de calcul	Modèle plus lourd, coût computationnel plus élevé	Avantage EfficientNetV2 en environnement CPU et itérations rapides
Dynamique d'apprentissage	Convergence rapide avec un bon rendement par epoch	Convergence plus progressive et régulière	EfficientNetV2 favorise des cycles d'expérimentation plus courts
Maîtrise et risque projet	Architecture plus récente mais désormais bien documentée	Architecture éprouvée et largement maîtrisée	Le Projet 9 introduit EfficientNetV2 comme une évolution maîtrisée, sans rupture brutale
Scalabilité et évolutivité	Scaling équilibré (profondeur, largeur, résolution)	Scalabilité plus limitée, montée en complexité coûteuse	EfficientNetV2 offre une base plus flexible pour des évolutions futures

Cette comparaison théorique met en évidence l'évolution apportée par EfficientNetV2 par rapport à VGG16, en termes de modernité et d'efficacité d'entraînement. Tout en conservant une base maîtrisée et comparable à la baseline du Projet 8, EfficientNetV2 permet d'accélérer les itérations et d'ouvrir la voie à une évaluation plus orientée performance. Cette évolution théorique sera désormais confrontée aux résultats observés sur les métriques de segmentation.

VERSUS MÉTRIQUES

Comparaison des métriques — UNet + VGG16 vs UNet + EfficientNetV2B0

Axe d'analyse	UNet + EfficientNetV2B0	UNet + VGG16	Lecture / impact sur la décision
Dice coefficient	≈ 0.62	≈ 0.59	EfficientNetV2B0 obtient un meilleur recouvrement global entre prédictions et vérités terrain, indiquant une segmentation globalement plus cohérente des zones d'intérêt.
IoU	≈ 0.23	≈ 0.22	L'amélioration reste modérée mais constante sur la métrique la plus exigeante, ce qui suggère une légère réduction des erreurs de bord et des faux positifs.
Pixel accuracy	≈ 0.62	≈ 0.59	Le gain observé est aligné avec Dice et IoU, sans indiquer de déséquilibre notable lié aux classes dominantes du jeu de données.
Score synthétique	≈ 0.49	≈ 0.47	L'avantage global en score synthétique reflète une performance plus équilibrée de EfficientNetV2B0 sur l'ensemble des métriques suivies.
Loss finale	≈ 0.38	≈ 0.41	Une loss plus faible traduit une meilleure optimisation globale du modèle, cohérente avec les gains observés sur les métriques de validation.
Dynamique d'apprentissage	Convergence rapide	Progression plus régulière	EfficientNetV2B0 atteint plus rapidement un niveau de performance élevé, tandis que VGG16 conserve une dynamique plus progressive et prévisible.

La comparaison des métriques met en évidence des performances globalement proches pour les deux modèles, entraînés dans des conditions strictement identiques afin de garantir une évaluation impartiale. EfficientNetV2B0 affiche néanmoins des scores légèrement supérieurs sur l'ensemble des indicateurs, ainsi qu'une convergence plus rapide. Ces résultats suggèrent qu'un backbone plus moderne permet d'améliorer la qualité de segmentation tout en conservant une dynamique d'apprentissage stable.

CONCEPTION STREAMLIT

Élément UX clé	Intention	Implémentation (extrait de code)	Effet utilisateur	Validation / accessibilité
Lisibilité élevée (zoom)	Garantir une lecture confortable à fort zoom	`css .block-container { max-width: 1200px; }` `font-size: 1.02rem; line-height: 1.55;`	Interface lisible à 200 % sans rupture de mise en page	Conforme aux bonnes pratiques d'accessibilité visuelle
Navigation simplifiée	Réduire la charge cognitive	`st.tabs(["Image RGB","Masque réel","Masque prédit"])`	Comparaison claire, une information à la fois	Parcours utilisateur guidé et intuitif
Accessibilité graphique	Éviter la dépendance à la couleur	`hatch="///"` / `hatch="xx"` `color="white", edgecolor="black"`	Graphiques compréhensibles en N&B ou daltonisme	Lecture indépendante des couleurs
Feedback utilisateur clair	Rendre l'état du système explicite	`with st.spinner("Prédiction…")` `st.success("Prédiction terminée.")`	Interaction rassurante et explicite	Réduction de l'ambiguïté utilisateur
Robustesse & erreurs lisibles	Éviter les blocages silencieux	`st.error("Erreur lors de l'appel API")` `st.stop()`	Compréhension immédiate des erreurs	Comportement prévisible et maîtrisé

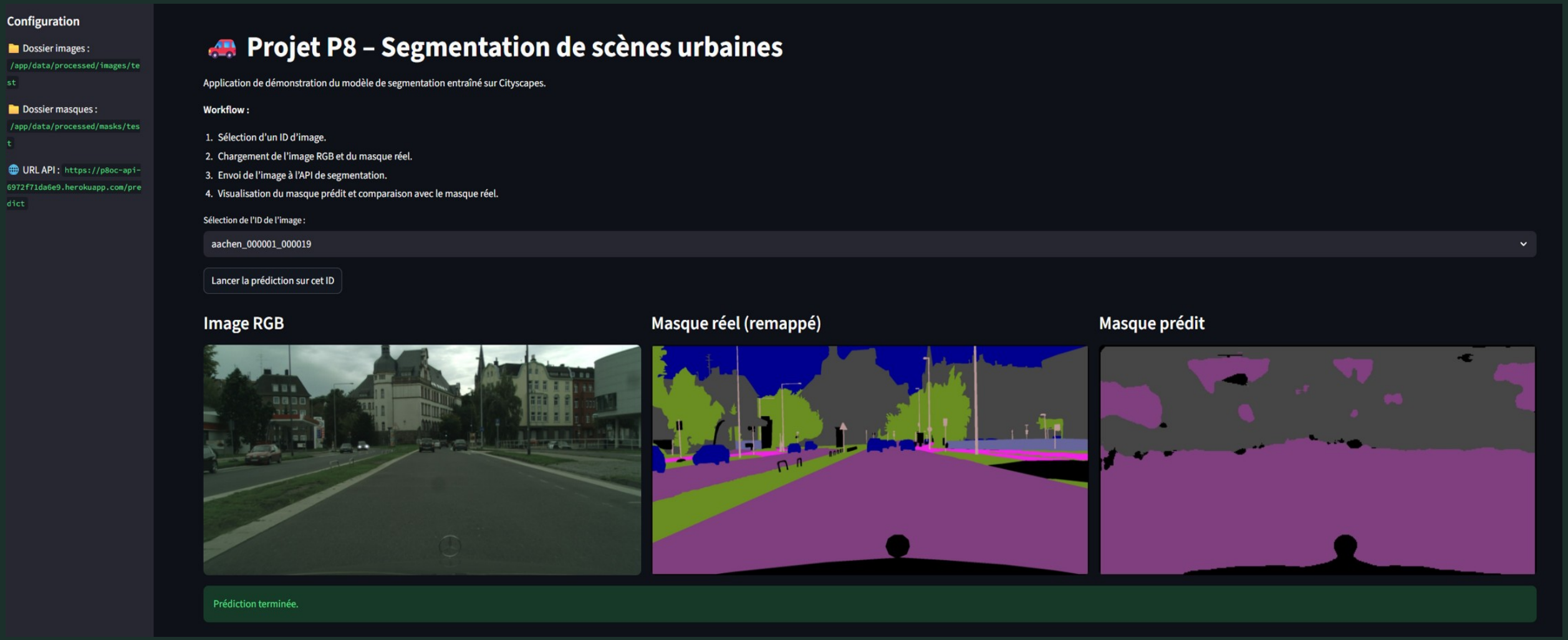
Le dashboard Streamlit intègre des choix de conception explicitement orientés accessibilité, directement au niveau du code. La mise en page et la typographie assurent une lisibilité stable à fort zoom, tandis que la navigation structurée et les visualisations non dépendantes de la couleur facilitent l'analyse. La gestion explicite des états et des erreurs renforce la robustesse et la fiabilité de l'interface dans un contexte de démonstration technique.

INTEGRATION HEROKU

Déploiement et orchestration – API et application Streamlit

Étape	Objectif	Implémentation clé	Insight technique
1. CI/CD GitHub Actions	Automatiser le pipeline de déploiement	Déclenchement sur `push`, exécution de `pytest`, jobs Heroku	Sécurise le processus de mise en production par validation continue
2. Déploiement de l'API	Maintenir une API stable en production	Job Heroku désactivé dans le workflow	Déploiement initial réalisé avec succès ; le job est ensuite désactivé en raison d'un *slug* dépassant 500 Mo, l'API reposant sur la dernière version active
3. Déploiement Streamlit	Mettre à jour l'interface web	Action `akhileshns/heroku-deploy`	Déploiement automatique de l'interface après validation
4. Point d'entrée unique	Centraliser l'exécution des services	Script `run.py`	Orchestration de l'API et du front-end via des variables d'environnement
5. Exécution de l'API	Servir le modèle de segmentation	`uvicorn main:app`	API FastAPI exécutée en mode CPU uniquement
6. Exécution Streamlit	Lancer l'interface utilisateur	`streamlit run app/streamlit_app.py`	Mise à disposition d'une démonstration interactive
7. Résultat final	Démonstration complète et fonctionnelle	API et interface utilisateur interconnectées	Chaîne de traitement ML déployée de bout en bout

DÉMONSTRATION



DÉMONSTRATION

CONCLUSION

Synthèse technique & perspectives — Projet 9 (POC segmentation)

Axe	Élément analysé	Mise en œuvre technique	Résultat observé	Lecture / enseignement
Synthèse	Données & tâche	Cityscapes (sous-ensemble), 256×512, remapping 34→8 classes, one-hot	Pipeline stable et reproductible	Cadre simplifié mais représentatif, adapté à une POC sur CPU
Synthèse	Architecture	UNet + EfficientNetV2B0 pré-entraîné, encodeur gelé	Intégration fluide dans UNet	Backbone moderne apportant efficacité sans rupture méthodologique
Synthèse	Performances	Dice ≈ 0.62, IoU ≈ 0.23, Acc ≈ 0.62, score synthétique ≈ 0.49	Légère amélioration constante vs VGG16	Gain qualitatif modéré mais cohérent sur toutes les métriques
Synthèse	Coût computationnel	CPU only, 7 epochs, batch size 4	Temps/epoch ≈ 4× plus faible	Accélération majeure du cycle d'itération à qualité comparable
Évolution	Optimisation du modèle	Backbone gelé, entraînement court	Capacité d'adaptation limitée	Fine-tuning partiel avec faible LR pour améliorer la spécialisation
Évolution	Qualité de segmentation	Résolution réduite, métriques globales	Détails fins partiellement limités	Augmenter la résolution et suivre Dice/IoU par classe
Évolution	Projection future	Architecture efficiente et compacte	Compatible production / embarqué	Quantification, pruning et mesure de latence pour déploiement

MERCI DE VOTRE ECOUTE

PROJET 9